Wireless Internet Of Things Lab

Versuchs-Nr.	Course Exercise 3
Versuchsname	Speed Measurement
Name	Muhammad Isma Haikal bin Ismail
Matrikelnummer	6057561
Versuchsdatum	06.11.2025
Betreuender Dozent	Prof. Dr.-Ing. Jens Werner

Task

For this exercise, we are required to write a program for the Node MCU ESP8266 module that reads the distance information from the ultrasonic sensor.

- The module and sensor will be placed on a trolley and manually moved towards a wall.
- The module shall calculate the momentarily speed from the variation of the distance measurement and document a list of relative time stamp, momentary speed and distance values.
- The data can be copied and pasted from serial monitor into an Excel sheet or directly fed into Excel by separate tool (e.g. PLX-DAQ).

Some optional tasks include:

- Send data via MQTT into node-red chart
- Create a graphical chart running on a web server on the EPS8266 module.
- For the trigger and time measurement of the ultrasonic pulse libraries provide already the required code. Write our own code without including these kinds of prebuilt libraries to trigger the sensor and calculate the distance.

Solution

In order to complete the task, the code must be written into visual studio code and connected to the esp 8266 using platformio. We also must download and configure both MQTT broker and node red for our optional tasks. After installing both applications, they must be set up so that they can be connected to each other and can send and receive data properly on the given IP. These are all steps during the configuration stage.

Firstly, the code is as follows and the workspace will be submitted alongside this report with the excel files for the sample data.

```cpp
#include <Arduino.h>
#include <ESP8266WiFi.h>
#include <PubSubClient.h>

const int TRIG_PIN = D5;  // HY-SRF05 Trig
const int ECHO_PIN = D6;  // HY-SRF05 Echo (5V via 2x150Ω series)

// ---------- WiFi & MQTT settings ----------
const char* ssid       = "Tplink";
const char* password   = "80436474877071282801";
const char* mqtt_server = "192.168.2.108";   // e.g. "192.168.1.10"
const int   mqtt_port   = 1883;
const char* mqtt_topic  = "esp/trolley";      // topic we will publish to

WiFiClient espClient;
PubSubClient client(espClient);

// ---------- Distance function ----------
float readDistanceCm() {
  // Trigger pulse
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  // Measure echo pulse width (µs)
  unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000UL); // 30 ms timeout
  if (duration == 0) return -1.0;

  // Convert to cm
  return duration / 58.0;
}

// ---------- Speed globals ----------
unsigned long lastTime = 0;
float lastDistance = -1.0;

// ---------- WiFi & MQTT helpers ----------
void setupWifi() {
  delay(10);
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
```

Image 1: Coding Part 1

```
  }

  Serial.println("");
  Serial.println("WiFi connected");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
}

void reconnect() {
  // Loop until connected
  while (!client.connected()) {
    Serial.print("Attempting MQTT connection...");
    // Create a random client ID
    String clientId = "ESP8266-Trolley-";
    clientId += String(random(0xffff), HEX);

    // Attempt to connect
    if (client.connect(clientId.c_str())) {
      Serial.println("connected");
      // (No subscriptions needed, we only publish)
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" try again in 5 seconds");
      delay(5000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  Serial.println("time_ms;distance_cm;speed_cm_per_s");

  setupWifi();
  client.setServer(mqtt_server, mqtt_port);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();   // keep MQTT connection alive

  unsigned long now = millis();
  float distance = readDistanceCm();
  if (distance < 0.0) {
    delay(50);
    return;
  }
```

Image 2: Coding Part 2

```
  float speed = 0.0;
  if (lastDistance >= 0.0) {
    unsigned long dt_ms = now - lastTime;
    if (dt_ms > 0) {
      float dt_s = dt_ms / 1000.0;
      speed = (distance - lastDistance) / dt_s;   // cm/s
    }
  }

  // ---- Serial CSV for Excel ----
  Serial.print(now);
  Serial.print(";");
  Serial.print(distance, 2);
  Serial.print(";");
  Serial.println(speed, 2);

  // ---- MQTT JSON payload ----
  // Example: {"t":1234,"d":47.25,"v":-13.22}
  String payload = "{";
  payload += "\"t\":";
  payload += now;
  payload += ",\"d\":";
  payload += String(distance, 2);
  payload += ",\"v\":";
  payload += String(speed, 2);
  payload += "}";

  client.publish(mqtt_topic, payload.c_str());

  lastTime = now;
  lastDistance = distance;

  delay(50);  // ~20 Hz
}
```

Image 3: Coding Part 3

The explanation for the coding is already provided by the comments on the coding itself using the comment function "//".

As discussed, the coding does not require any additional library to be included. The only library included was from the MQTT broker.

The setup for the ESP is as follows.

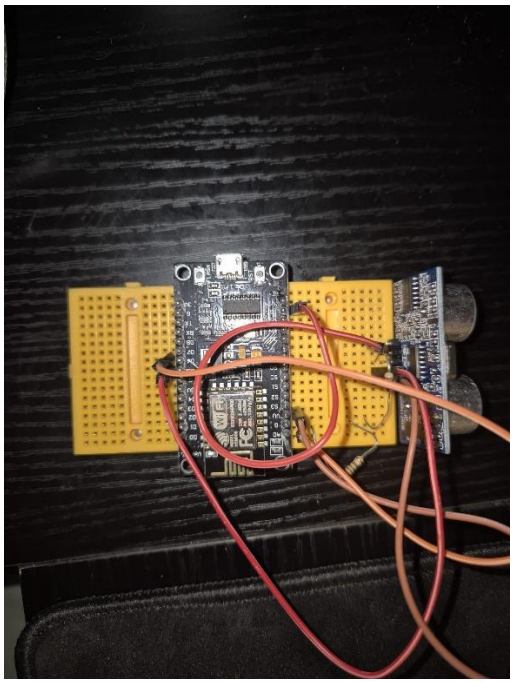
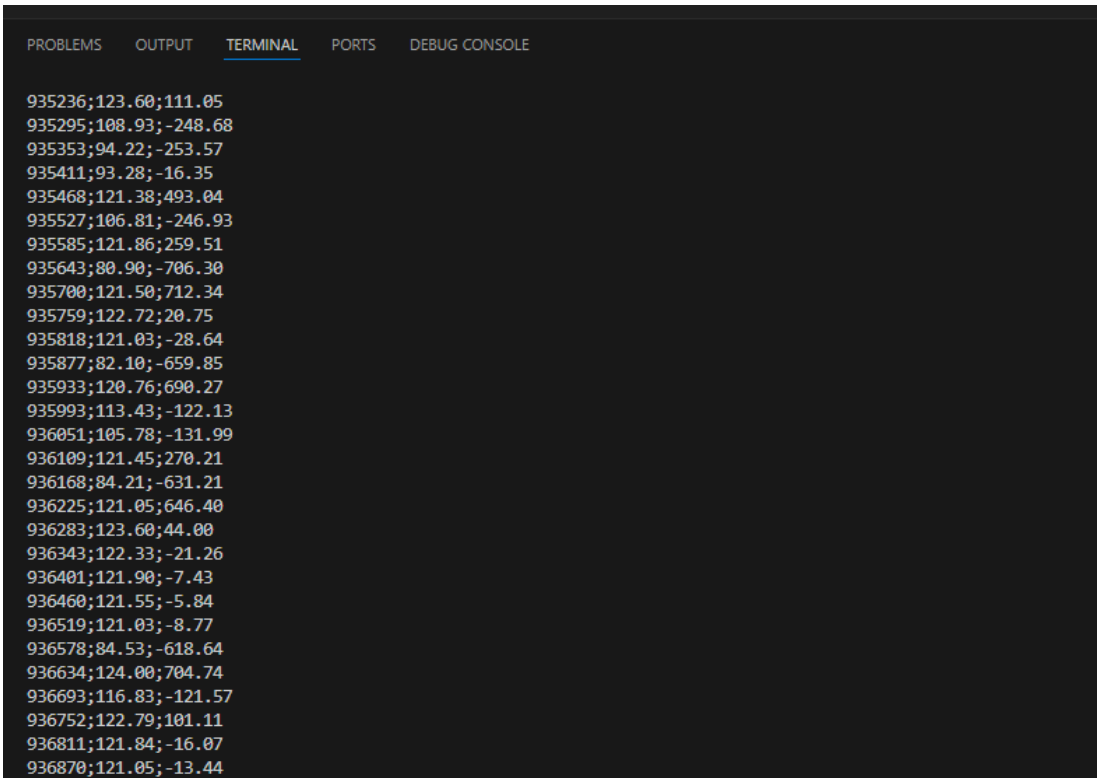


Image 4: Setup for ESP and Ultrasonic Sensor

These coding allows the output to be copied and directly pasted into excel so that a proper graph can be plotted.



PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE

```
935236;123.60;111.05
935295;108.93;-248.68
935353;94.22;-253.57
935411;93.28;-16.35
935468;121.38;493.04
935527;106.81;-246.93
935585;121.86;259.51
935643;80.90;-706.30
935700;121.50;712.34
935759;122.72;20.75
935818;121.03;-28.64
935877;82.10;-659.85
935933;120.76;690.27
935993;113.43;-122.13
936051;105.78;-131.99
936109;121.45;270.21
936168;84.21;-631.21
936225;121.05;646.40
936283;123.60;44.00
936343;122.33;-21.26
936401;121.90;-7.43
936460;121.55;-5.84
936519;121.03;-8.77
936578;84.53;-618.64
936634;124.00;704.74
936693;116.83;-121.57
936752;122.79;101.11
936811;121.84;-16.07
936870;121.05;-13.44
```

Image 5: Data Output on serial monitor

```
mosquitto.exe  -v -c minimal.conf
```

Using this command on the command prompt allows us to view if the MQTT is connected to our IP Address and ESP already and shows some data as evidence.

Node-Red Setup:

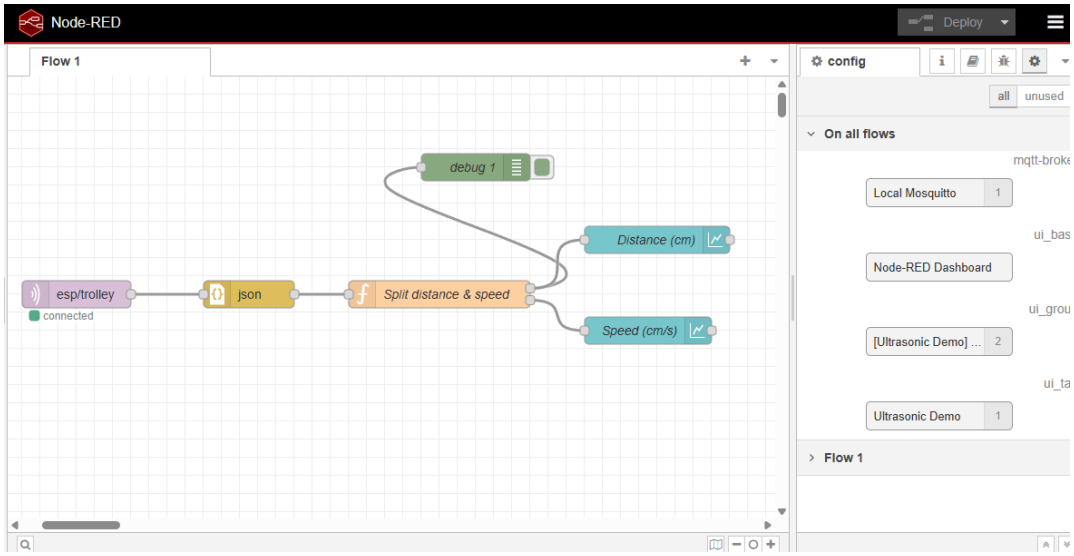


Image 5: Flows on Node-Red

On Node-Red, the flow is set up exactly as above. The first node is an MQTT in node to receive the data from esp. Then, a json node is utilized to transform the pure data received from MQTT into an appropriate format for the User Interface. Afterwards, we use the function node to plot a graph using the sets of data provided. The chart node is finally inserted, separated into both speed and distance graph to be displayed on the UI.

Results

The results are as follows:

The data on serial monitor is copied and pasted on excel, and a graph is then plotted.

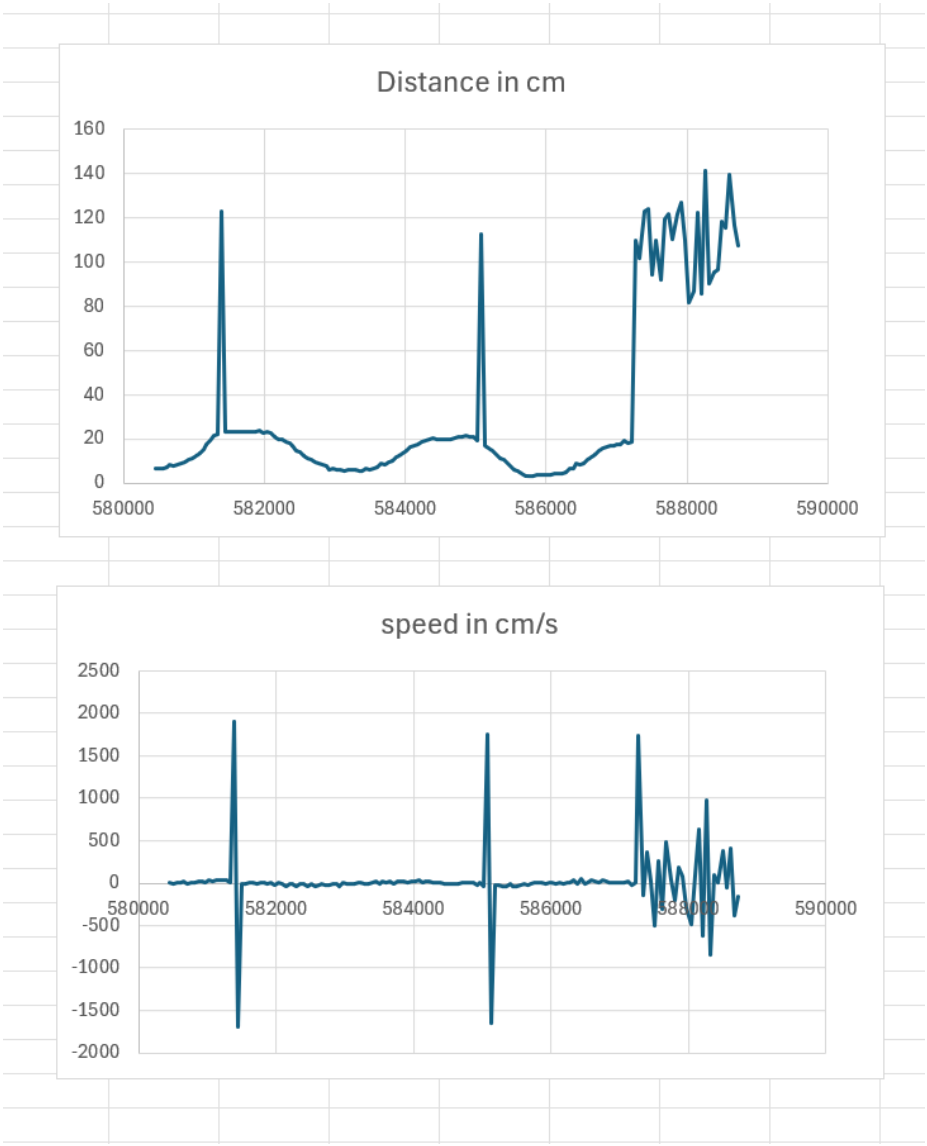


Image 6: Excel Graph using Data from Serial Monitor

Afterwards, the data from Node-Red is also obtained on the UI.

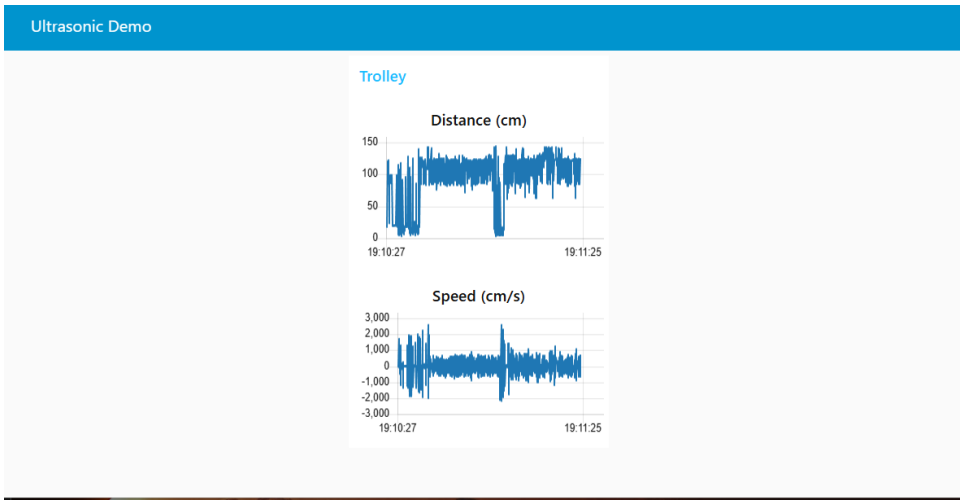


Image 7: Node Red UI

These sets of data are obtained from when the ESP is moved slowly back and forth against a wall. Therefore, the results shown in the graph are as expected.

Conclusion

After careful consideration, the exercise is seen as completed as it has completed most if not all the requirements provided by the task. The results are within the expected range although with some inconsistencies. The setup is repeatable in other environments as long as the Wi-Fi used to connect the MQTT and ESP are on 2.4G wavelengths. This is because ESP 8266 can only be connected to these wavelengths. Some improvements that can be applied are using resistors of higher Resistance such as 1 kOhm and 2 kOhm. Since the Ultrasonic sensor could function to its full potential when provided with the exact Voltage or Current. We also require the IP address of the Wi-Fi used, so that ESP and MQTT as well as Node-Red can all be connected seamlessly.